ROOTSBLOOM

Website Improvement & Bug-Fix Blueprint

Antigravity-ready implementation brief

Website: https://rootsbloom.netlify.app/
Repository: https://github.com/Pradecer/Roots_Bloom

Scope: UI/UX • Frontend bugs • Commerce readiness • Performance • Accessibility • Security review checklist
Explicit exclusion: Login, signup, authentication and password flows

Prepared from repository/source review • Login & authentication intentionally excluded

1. Is document ka purpose

Is document mein RootsBloom website ke source-code review ke dauran identify hui problems, unka business/user impact, recommended fixes aur test/acceptance criteria diye gaye hain. Isse Antigravity mein upload karke implementation brief ke roop mein use karein.

Important limitation: yeh repository aur frontend source ka audit hai, complete live penetration test, Lighthouse run, backend/API assessment ya payment-provider verification nahi. Jahan backend/security behaviour source se confirm nahi ho saka, use 'verify' ke roop mein mark kiya gaya hai—confirmed vulnerability ke roop mein nahi.

Audit snapshot

Provisional source/product-readiness score	**4.5 / 10**
Visual structure	**6 / 10**
Catalog data quality	**3 / 10**
Product discovery	**5 / 10**
Accessibility	**4 / 10**
Commerce readiness	**3 / 10**
Frontend architecture	**5 / 10**

Score source-code/product-readiness review ka qualitative estimate hai; yeh measured security score ya Lighthouse performance score nahi hai.

2. Priority guide

- CRITICAL: Real payment/order flow ko affect kar sakta hai; production launch se pehle resolve karein.
- HIGH: Trust, data integrity, order fulfilment ya core shopping experience par major impact.
- MEDIUM: Discovery, accessibility, usability ya consistency issue.
- LOW / VERIFY: Polish, optimization opportunity, ya aisa claim jise operational evidence se verify karna hai.

3. Detailed findings and fixes

RB-001 — Checkout order creation is frontend-only in inspected flow

Priority: CRITICAL | Location: src/pages/CheckoutPage.jsx; src/context/AppContext.jsx

Problem: Checkout handler placeOrder(paymentMethod) call karta hai. Inspected AppContext flow local React state mein order object banata hai, orders state update karta hai, cart clear karta hai aur success page par navigate karta hai. Is flow mein actual payment gateway transaction/verification nazar nahi aayi.

Why it matters: Production mein user ko order/payment success dikh sakta hai bina verified payment ya durable server-side order ke. Refresh/browser change par order ka record bhi lost ho sakta hai.

Recommended fix: Backend par order create endpoint banayein. Server par product IDs, current prices, discounts, stock, delivery fee, address aur total dobara calculate/validate karein. UPI/card ke liye trusted payment provider integrate karein; payment success sirf server-side webhook/signature verification ke baad mark karein. COD order ko 'COD confirmed' status dein, 'paid' nahi. Idempotency key aur duplicate-submit protection add karein.

Done when: Tampered client total accept na ho; payment success page sirf backend-confirmed order dikhaye; repeated request duplicate order/payment create na kare.

RB-002 — Order tracking is a demo simulator, not reliable live tracking

Priority: HIGH | Location: src/pages/OrderTrackingPage.jsx

Problem: Code comment/logic demo simulator indicate karta hai. currentStatusIndex aur handleSimulateNextStep tracking step ko increment karte hain; modulo logic se Delivered ke baad status phir Order Placed ho sakta hai. Dates/times hardcoded hain.

Why it matters: Customer ko false delivery status, impossible status cycle aur misleading timestamps dikh sakte hain.

Recommended fix: Production UI se simulate-next-step control aur fake timestamps hataayein. Backend/carrier events se tracking timeline render karein. Status progression server-side validate ho: placed → confirmed → packed → shipped → out for delivery → delivered; cancelled/returned jaise terminal/alternate states define karein. Real event na ho to 'Tracking update abhi available nahi' dikhayein.

Done when: Delivered status terminal ho; timestamps actual backend events se aayen; demo mode clearly labelled ho aur production build mein disabled ho.

RB-003 — All products show identical 4.8 rating and 64 reviews

Priority: HIGH | Location: src/data/catalog.js

Problem: Source data mein 109 products ke liye rating average 4.8 aur review count 64 same hai. Reviewer names bhi repeat hote hain (e.g. Ritu Verma, Amit K. across catalog).

Why it matters: Yeh misleading social proof ho sakta hai, customer trust aur consumer-protection risk create karta hai.

Recommended fix: Fake/static review data ko production catalog se remove karein. Real review records ko productId se link karein; average/count backend se calculate karein. Verified-purchase badge sirf delivered order se linked review ko dein. Reviews nahi hain to 'No reviews yet' / 'Be the first to review' show karein.

Done when: Har product ka rating/count actual review records se derived ho; empty review state honest ho; sample/demo reviews production mein visible na hon.

RB-004 — Free delivery and COD promises are hardcoded on every product card

Priority: HIGH | Location: src/components/ProductCard.jsx

Problem: Product card par 'Free Delivery • COD Available' static text hai, eligibility/price/pincode ke hisaab se conditional nahi.

Why it matters: Customer checkout par delivery fee ya COD unavailability dekh sakta hai, jis se surprise charges aur cart abandonment ho sakta hai.

Recommended fix: Shipping/COD eligibility ko central service/config se derive karein. Product card par universal promise ke bajay 'Delivery options at checkout' ya location-aware estimate dikhayein. Pincode, product restrictions, cart value aur fulfilment rules ke basis par COD/free delivery show karein.

Done when: Card, cart aur checkout par delivery/COD claims ek hi source of truth se aayen; ineligible order par promise na dikhe.

RB-005 — Product cards are clickable divs instead of semantic links

Priority: MEDIUM | Location: src/components/ProductCard.jsx

Problem: Whole card div par onClick navigation hai. Native anchor/Link semantics aur keyboard behaviour nahi milta.

Why it matters: Keyboard-only users, screen readers aur open-in-new-tab/browser navigation ke liye interaction weak ho sakta hai.

Recommended fix: Product title/image ko React Router Link/anchor banayein. Add-to-cart/wishlist actions ko separate button rakhein; nested interactive controls ko clickable parent ke andar ambiguous na banayein. Visible focus style aur meaningful accessible names add karein.

Done when: Tab/Enter se product open ho; buttons independently operate karein; link context menu/open in new tab work kare.

RB-006 — Homepage carousel auto-advances without adequate user control

Priority: MEDIUM | Location: src/pages/HomePage.jsx

Problem: Carousel roughly every 5 seconds auto-advance karta hai; pause control, previous/next controls, focus pause aur reduced-motion handling ka evidence nahi mila. Hero image alt generic hai.

Why it matters: Motion-sensitive users aur keyboard/screen-reader users ke liye experience difficult ho sakta hai; promotional content miss ho sakta hai.

Recommended fix: Previous/next buttons, pause/play control, slide indicator aur accessible labels add karein. Keyboard focus/hover par autoplay pause karein; prefers-reduced-motion respect karein. Alt text ko actual banner purpose ke mutabik likhein. Auto-rotation ko disable karna bhi acceptable option hai.

Done when: User carousel pause/advance kar sake; reduced-motion users ke liye autoplay off ho; controls screen reader se understandable hon.

RB-007 — Cart does not enforce stock limits in addToCart logic

Priority: HIGH | Location: src/context/AppContext.jsx

Problem: addToCart item add/increment karta hai, lekin source review mein authoritative stock validation nahi dikhi.

Why it matters: Out-of-stock ya quantity-limit se zyada items cart/order mein ja sakte hain. Client-side stock display ko manipulate bhi kiya ja sakta hai.

Recommended fix: Server-side inventory reservation/validation add karein, cart update aur order placement dono par. Client-side check sirf UX convenience ho. Quantity limit, race condition aur stock-changed error state handle karein.

Done when: Stock 0 par add/order reject ho; simultaneous purchases oversell na karein; checkout par latest stock validate ho.

RB-008 — Orders are stored only in React state

Priority: HIGH | Location: src/context/AppContext.jsx

Problem: Orders useState mein maintain hote hain; inspected flow mein persistent backend order store nahi dikha.

Why it matters: Refresh, new browser session ya device change par order history disappear ho sakti hai.

Recommended fix: Orders ko backend database mein persist karein. User ke authorized order-history endpoint se fetch karein. Loading, empty, error aur retry states add karein. PII ko browser localStorage mein unnecessary store na karein.

Done when: Refresh ke baad valid order history available ho; data server se aata ho; errors gracefully handled hon.

RB-009 — Delivery fee calculation conflicts with product-card messaging

Priority: MEDIUM | Location: src/context/AppContext.jsx; src/components/ProductCard.jsx

Problem: AppContext mein delivery fee cartSubtotal >= ₹500 ya zero subtotal par free, otherwise ₹70 hai; ProductCard har item par Free Delivery batata hai.

Why it matters: Product listing aur checkout ke beech price expectation mismatch hota hai.

Recommended fix: Shipping calculation ko shared utility/backend quote service mein centralize karein. Cart, product page, mini-cart aur checkout same result dikhayein. Threshold, fee, pincode exceptions aur shipping promotions configurable hon.

Done when: Same cart/pincode ke liye har screen par same shipping amount/eligibility aaye; threshold messaging exact ho.

RB-010 — Homepage hero CTA destinations may be too narrowly mapped

Priority: MEDIUM | Location: src/pages/HomePage.jsx

Problem: Hero CTA product detail par direct link karta hai; kuch campaign/category slides ko collection/category landing page par jaana chahiye ho sakta hai.

Why it matters: Campaign ka context aur discovery path lose hota hai; user ko sirf ek product par bhejna har banner ke liye suitable nahi.

Recommended fix: Banner data model mein explicit destination type + URL rakhein: product, category, collection ya campaign. CTA label aur destination match karein. Invalid/missing destination ke liye safe fallback.

Done when: Har slide ka CTA intended destination par jaye; no dead links; mobile/desktop same destination.

RB-011 — 10 product discount percentages do not match listed prices

Priority: MEDIUM | Location: src/data/catalog.js

Problem: Source comparison mein 10 products ke declared discount percentage aur base/offer price se calculate hone wale discount mein mismatch mila. Example: ₹400 se ₹350 par discount 12.5% hai, rounded display 13% hona chahiye, 12% nahi.

Why it matters: Discount label inaccurate ho sakta hai aur pricing transparency/trust impact hota hai.

Recommended fix: Discount percentage manually store na karein; derive karein: Math.round(((basePrice - offerPrice) / basePrice) * 100). Validate basePrice > 0, offerPrice <= basePrice; invalid data ko build/test mein fail karein. Currency formatting centralized ho.

Done when: Catalog ke sabhi discounts displayed price math se match karein; invalid pricing record CI test fail kare.

RB-012 — Every product gallery has only one image; all primary images are PNG

Priority: MEDIUM | Location: src/data/catalog.js and product image assets

Problem: Source inventory mein 109/109 primary images PNG aur 109/109 product galleries mein sirf ek image mili.

Why it matters: Product texture, scale, packaging aur details judge karna mushkil; PNG files unnecessary large ho sakti hain.

Recommended fix: Product gallery mein multiple genuine views add karein (front, side, close-up, scale/lifestyle where available). Images ko AVIF/WebP responsive sizes mein convert karein; width/height reserve karein; thumbnails, zoom and swipe support add karein. Original source assets preserve karein.

Done when: Key products par multiple views hon; image dimensions/layout shift controlled ho; modern formats supported hon with fallback.

RB-013 — Best Sellers section renders the full 109-product catalog

Priority: MEDIUM | Location: src/pages/HomePage.jsx

Problem: Best Sellers area mein PRODUCTS.map(...) se catalog ka full set render hota hai; curated best-seller selection nahi.

Why it matters: Homepage overly long/heavy hota hai; user ko priority products aur categories samajhna mushkil.

Recommended fix: Homepage par 4–8 curated products show karein. Real sales/order data available hone par best-seller ranking derive karein; data na ho to 'Featured Picks' label use karein. View all link ko category/collection page par bhejein.

Done when: Homepage section limited curated count show kare; fake best-seller claims na hon.

RB-014 — Devotional category taxonomy/filter mismatch

Priority: MEDIUM | Location: src/data/catalog.js and category filter configuration

Problem: Category config devotional key use karta hai, lekin Shiv Tray Garden aur Buddha Tray Garden jaise products ka main category 'Tray Gardens' hai aur devotional identity tags/subcategory mein hai. Main-category-only filter in products ko Devotional page se omit kar sakta hai.

Why it matters: Products expected category mein nahi milte; filters confusing aur incomplete lagte hain.

Recommended fix: Product taxonomy define karein: primary category + secondary tags/collections/occasion. Category page filter ko intended taxonomy field par query karein. Devotional collection tag se relevant tray-garden products include karein, without duplicate category hacks.

Done when: Devotional collection expected products include kare; unrelated tray gardens exclude hon; filter/search behaviour documented ho.

RB-015 — Product taxonomy mixes product types, occasions and gifting themes

Priority: MEDIUM | Location: src/data/catalog.js; category navigation

Problem: Catalog mein plants, miniatures, bird feeders, decor/furniture-style items, gifts aur devotional products ek hi category hierarchy mein mix hain.

Why it matters: User ko browse karne ka clear mental model nahi milta; category filters aur SEO landing pages weak ho sakte hain.

Recommended fix: Navigation ko two dimensions mein organize karein: 'Shop by Product' (e.g. terrariums, planters, miniatures, bird feeders, decor) aur 'Shop by Occasion/Theme' (gifting, devotional, housewarming, etc.). Product ko one primary category aur multiple tags assign karein.

Done when: Category labels mutually understandable hon; product type aur occasion alag filters/collections hon.

RB-016 — Image delivery and performance need optimization

Priority: MEDIUM | Location: src/data/catalog.js; image assets; HomePage

Problem: All catalog primary images PNG hain. Source inspection se actual transfer size/Lighthouse score measure nahi hua, lekin image payload optimize karne ka clear opportunity hai.

Why it matters: Large images mobile network par page load aur scrolling ko slow kar sakti hain.

Recommended fix: AVIF/WebP responsive srcset/sizes use karein; below-the-fold images lazy-load karein; hero/LCP image ko selectively preload karein; explicit width/height ya aspect-ratio dein; thumbnails use karein; CDN/cache headers configure karein. Har image ko blindly preload na karein.

Done when: Mobile throttling par image bytes reduce hon; LCP/CLS metrics measured hon; hero image priority aur below-fold lazy loading verify ho.

RB-017 — Brand/design system differs from PRD tokens

Priority: LOW | Location: src/index.css and project PRD/design references

Problem: PRD aur CSS tokens mein green, accent, background aur border-radius values alag hain. Example: PRD primary green #2F6B4F vs CSS #22533C; accent #E8703C vs #C85A32; PRD card radius 6–8px vs CSS 4px; background white vs #FAF7F2.

Why it matters: Design decisions inconsistent ho sakte hain; future components mein visual drift badhega. Difference apne aap bug nahi—intentional design choice bhi ho sakta hai.

Recommended fix: Ek final design direction approve karein (warm botanical/premium editorial look is a reasonable direction), phir CSS variables/tokens ko single source of truth banayein. PRD ko actual chosen tokens se update karein. Buttons, cards, forms, spacing, typography aur states ke reusable tokens define karein.

Done when: PRD, CSS variables aur component library same approved tokens use karein; duplicate hardcoded colors/radii reduce hon.

RB-018 — Trust claims and promotional copy need operational verification

Priority: VERIFY | Location: src/App.jsx; src/components/Header.jsx; src/components/TrustBar.jsx

Problem: Source mein Enterprise-Grade / Anti-Cartoon Compliant jaise unusual footer claims, 20,000+ pincodes, 7-day free replacement aur promo banner ₹100 off with FESTIVE200 jaise claims/text mile.

Why it matters: Agar claims operationally accurate nahi ya promo terms unclear hain, brand credibility aur customer expectations damage ho sakte hain.

Recommended fix: Har claim ko business owner/operations se verify karein. Unsubstantiated claims remove karein. Replacement policy mein eligibility, exclusions, time window aur process link karein. Promo code, discount amount,

minimum cart, expiry aur terms consistent rakhein. 'Anti-Cartoon Compliant' jaisi confusing copy ko meaningful trust proof se replace karein.

Done when: Har public claim documented and current ho; promo banner aur checkout discount same rules use kare.

RB-019 — WhatsApp/contact details are hardcoded in application source

Priority: VERIFY | Location: src/App.jsx

Problem: WhatsApp contact detail source mein hardcoded hai.

Why it matters: Number change hone par code release chahiye; wrong/placeholder contact customer support ko break kar sakta hai.

Recommended fix: Contact details ko environment/config/CMS-backed setting mein rakhein. Production build mein placeholder validation add karein; correct country code, privacy and consent expectations check karein.

Done when: Production contact number verified ho; placeholder/old number release pipeline mein detect ho.

4. Security / pentest checklist (login/authentication excluded)

Neeche ke points source review se confirmed vulnerabilities nahi hain. Inhe authorized staging environment par verify karna hai. Production data ko modify/delete kiye bina, low-rate, non-destructive checks karein.

- **SEC-01 — Client-side secrets:** Vite env variables aur built JS bundle inspect karein. Secret API keys, private tokens, payment secrets ya credentials browser bundle mein nahi hone chahiye. Public publishable keys only if provider explicitly permits.
- **SEC-02 — Order ID access control (non-login scope):** Agar order lookup/tracking endpoint hai, check karein ki predictable order ID se doosre customer ka address/phone/order details expose na ho. Authorization/tokenized access server-side enforce ho.
- **SEC-03 — Price/discount tampering:** Staging par browser devtools se client subtotal, discount, shipping fee aur product price alter karke checkout request test karein. Server ko client-supplied total trust nahi karna chahiye; server recompute kare.
- **SEC-04 — Inventory/order race conditions:** Parallel add/order requests se negative stock, overselling ya duplicate order/payment create na ho. Atomic database transaction/stock reservation use karein.
- **SEC-05 — XSS and unsafe rendering:** Product descriptions, review text, search terms aur URL parameters ko test strings se verify karein. React escaping preserve karein; dangerouslySetInnerHTML avoid/sanitize karein. Actual exploit payloads production mein run na karein.
- **SEC-06 — CORS and API exposure:** Backend API ke allowed origins, methods, credentials aur error responses review karein. Wildcard credentialed CORS avoid karein; only required origins allow karein.
- **SEC-07 — HTTP security headers:** HTTPS, HSTS, Content-Security-Policy, X-Content-Type-Options, Referrer-Policy, frame protection (CSP frame-ancestors / X-Frame-Options) verify karein. CSP ko report-only se tune karke enforce karein.
- **SEC-08 — Source maps and build artifacts:** Production deployment mein source maps publicly exposed hain ya nahi check karein. Public source maps policy decision ho; secrets kabhi source map mein na hon.
- **SEC-09 — Dependency hygiene:** npm audit / lockfile review, outdated and vulnerable packages, Vite/React build dependencies check karein. Upgrade with tests; blindly force-upgrade na karein.
- **SEC-10 — Rate limiting and abuse controls:** Order creation, coupon validation, review submission, tracking lookup and contact endpoints par rate limiting, bot controls and abuse logging verify karein.
- **SEC-11 — PII and logging:** Address, phone, payment references ko console logs, analytics events, URLs ya error reports mein leak na karein. Data retention and access minimization follow karein.
- **SEC-12 — Payment webhook verification:** Payment provider webhook signature, amount/currency/order ID matching, replay/idempotency and event ordering server-side verify karein. Client redirect alone ko payment proof na maanein.

5. UI/UX recommended homepage structure

1. Header: logo, clear primary navigation, search, account/cart icons; promo strip with exact and valid offer terms.
2. Hero: one clear value proposition, one primary CTA and one secondary CTA; destination campaign/category/product ke mutabik.
3. Shop by category: 5–7 understandable product-type categories with consistent imagery.
4. Shop by occasion/theme: gifting, devotional, housewarming etc. as separate collections/tags.
5. Featured Picks: 4–8 curated products; 'Best Sellers' label only when real sales data supports it.
6. Trust section: only verified delivery, replacement and support claims with policy links.
7. Reviews: genuine product-specific review snippets, or omit until real reviews exist.
8. Footer: concise contact, shipping/returns/privacy/terms links; remove meaningless or unverifiable badges.

6. Implementation order (recommended)

Phase 1 — Commerce correctness (launch blocker)

RB-001, RB-002, RB-007, RB-008, RB-009. Build backend order/payment/inventory/tracking truth; do not fake success states.

Phase 2 — Trust and data integrity

RB-003, RB-004, RB-011, RB-018, RB-019. Correct reviews, shipping promises, pricing, policy and contact data.

Phase 3 — Discovery and IA

RB-013, RB-014, RB-015, RB-010. Curate homepage and repair taxonomy/collections/CTAs.

Phase 4 — Accessibility and visual system

RB-005, RB-006, RB-017. Semantic interactions, carousel controls, approved design tokens.

Phase 5 — Media/performance and SEO

RB-012, RB-016. Product gallery depth, responsive modern formats, loading strategy, metadata and Core Web Vitals measurement.

Phase 6 — Security verification

SEC-01 to SEC-12. Run only in authorized staging; document evidence, severity, reproduction steps and remediation.

7. Antigravity ke liye copy-paste master instruction

Neeche ka prompt Antigravity mein is document aur repository ke saath use karein:

```
Act as a senior React/Vite engineer, e-commerce architect, accessibility specialist and application security reviewer. Read this entire RootsBloom audit document and inspect the current repository before changing anything. Implement the listed findings in priority order, excluding login, signup, authentication, password reset and identity flows entirely. Do not assume a backend exists: first map the current architecture and clearly identify which requirements need backend/API/database/payment-provider support. Never simulate real payment, order confirmation, stock, reviews or shipment tracking in production. If backend credentials/services are not available, build safe integration interfaces and honest unavailable/pending UI states rather than fake success. Preserve the existing brand identity where reasonable, but establish one consistent design-token source of truth. Make changes in small phases; before each phase explain files to change and risks. Do not delete working features or rewrite the whole app unnecessarily. Add automated tests for price/discount calculations, shipping fee consistency, stock validation, category filtering, order status progression, and accessible product-card/carousel behaviour. Run lint, tests and production build after changes; fix regressions. Check mobile and desktop layouts. For security, perform only non-destructive checks in an authorized staging environment; do not attack third-party systems, access real customer data, or run destructive tests. Do not claim a security issue is fixed unless you can show the code change and a test/evidence. At the end provide: (1) completed findings with file paths, (2) items blocked on backend/provider/business input, (3) tests/build results, (4) remaining risks, and (5) exact manual QA steps. Keep login/authentication out of scope.
```

8. Files to inspect first

File	Review focus
src/pages/CheckoutPage.jsx	Checkout/payment flow

src/context/AppContext.jsx	Cart, orders, shipping and state
src/pages/OrderTrackingPage.jsx	Tracking simulation/statuses
src/data/catalog.js	Product data, prices, categories, ratings, image galleries
src/components/ProductCard.jsx	Card interaction, shipping/COD claims
src/pages/HomePage.jsx	Hero carousel and homepage product sections
src/App.jsx	Global layout, footer claims, contact
src/components/Header.jsx	Promo banner and offer copy
src/components/TrustBar.jsx	Delivery/replacement trust claims
src/index.css	Design tokens and global styles

9. Final QA checklist

- ☐ Checkout: total server-side calculate ho; payment confirmation verified ho; duplicate order na bane.
- ☐ Cart: stock, quantity, shipping and coupon rules consistent hon.
- ☐ Orders: refresh ke baad order history persist ho; correct status show ho.
- ☐ Tracking: fake timestamps/statuses nahi; Delivered ke baad status reverse na ho.
- ☐ Catalog: ratings/reviews genuine; discount math correct; category filters complete.
- ☐ Product detail: multiple images, keyboard-accessible gallery and clear delivery information.
- ☐ Homepage: curated products; working CTAs; carousel pause/controls/reduced motion.
- ☐ Responsive: 360px mobile, tablet and desktop par no horizontal overflow or clipped content.
- ☐ Accessibility: keyboard navigation, visible focus, semantic links/buttons, alt text, contrast, labels.
- ☐ Performance: responsive image formats, lazy loading, dimensions, measured LCP/CLS/INP.
- ☐ Security: no client secrets; price/order checks server-side; headers/CORS/rate limits/webhooks reviewed.
- ☐ Trust: promo, shipping, replacement, contact and review claims verified and consistent.

10. Source files / repository

Repository: https://github.com/Pradecer/Roots_Bloom

Live website: https://rootsbloom.netlify.app/

Audit note: Findings are based on source inspection discussed in this review. Re-check current branch/version before implementation, because repository contents may have changed since the review.